Tutoring 03
Logging & Servlets

Francesco L. De Faveri

Web Applications Tutoring
Academic Year: 2025-2026

Università degli Studi di Padova
Dipartimento di Ingegneria dell'Informazione

Outline

- General Information
- Project Structure
- Logs
- From DB to Servlets

General Information

Equally Commit

You should have set up everything with your bitbucket repository.

Suggestion:

- Create the .gitignore and commit it. (Try to create a TODO file for each member and exclude it from the commit)
- Create the README.md (**each group member should add its name and some information about the project**)

Do it using git and not the bitbucket browser interface.

This way you are sure that you configured everything properly.

Remark on Tutorship Code

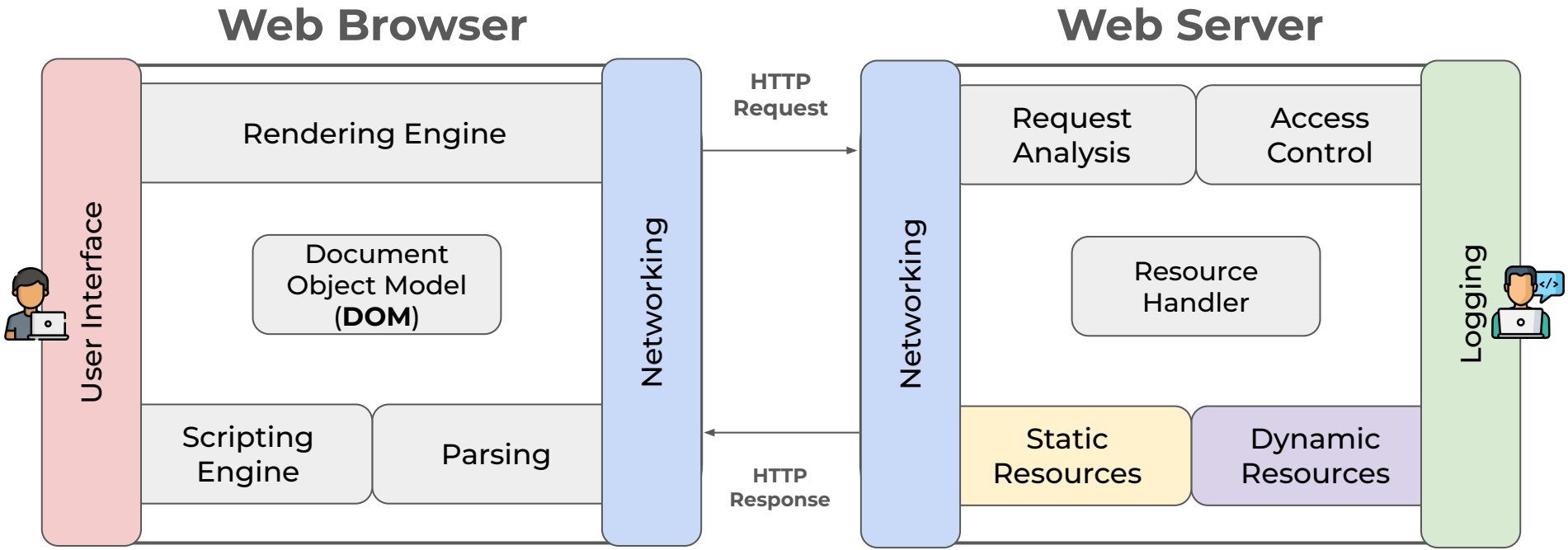
The tutorship Code repository (2026, 2025, etc...) (nor the prof repository) are **NOT THE CODE OF YOUR PROJECT!**

Your homework must be based on **original content produced by the group** - code, report, and slides.

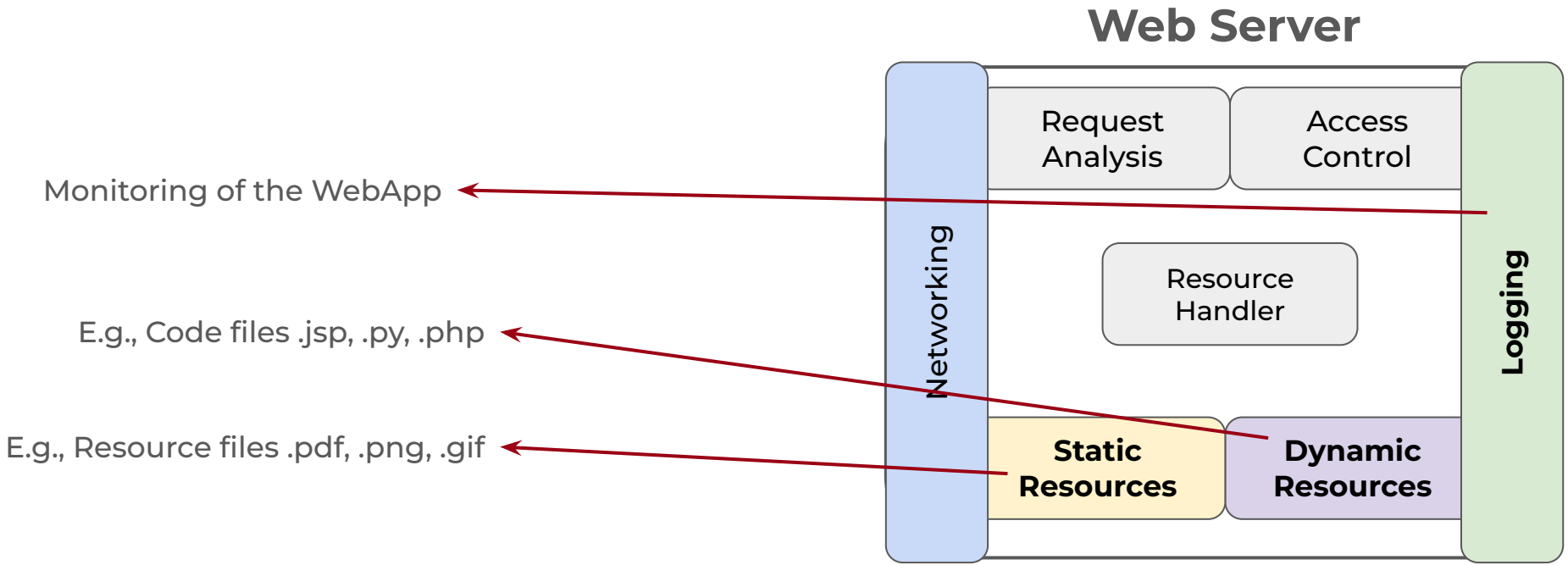
You can look on the repositories to see how some problems have been solved and how we implemented some solutions, **BUT YOU SHOULD NOT JUST COPY AND PASTE IT** for your HWs.

Project Structure

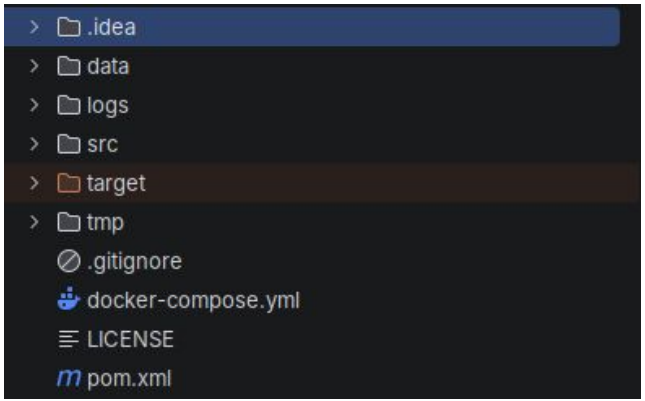
Browser and Server Architecture



Browser and Server Architecture



Project Structure



data → contains the DB data (if you use docker)

logs → contains the log of your web application (if you use docker)

src → contains all the code of your web application

target → contains all the files generated by MAVEN (including the .war file)

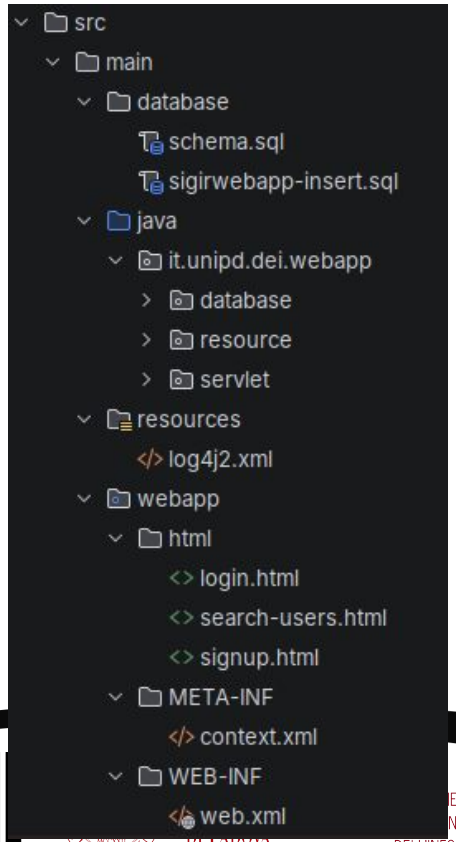
tmp → contains some “configuration files” for docker (if you use it)

.gitignore → contains the files/patterns that git should ignore

docker-compose → allows you to containerize your web application

pom → contains the MAVEN dependencies, etc.

Project Structure: src



database → contains your sql scripts

java → contains all the java code

resources → contains configuration files

webapp → contains some **configuration files** (WEB-INF and META-INF folders) and the **presentation layer**: HTML, css, javascript (We'll see in future lectures).

Logs

Why Logs?

Logs are **records of events or messages generated by the web app**.

They should help both to understand what your web app is doing and to detect errors.

Remember that there exist different levels of logs and have a different levels of severity: info, warn, error.

Logs may contain only info about accesses (i.e., logged users) or more detailed such as forms, and actions details.

Suggestion: in your web application try to log everything.

Log4j

Log4j is a logging framework that supports logging in java.

The **log4j.xml file is a configuration file** used by the Log4j2 logging framework to define how log messages should be processed and printed.

It must be placed in the “**resources**” folder.

log4j.xml

```
<Configuration status="INFO" monitorInterval="0" name="log4j2-config">
    <Appenders>
        <RollingRandomAccessFile name="RFILE" fileName="${sys:catalina.base}/webapps/my-logs/sigir-webapp.log"
                                 filePattern="${sys:catalina.base}/webapps/my-logs/$${date:yyyy-MM}/sigir-servlets-%d{yyyyMMdd}-%i.log.gz">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="250 MB"/>
            </Policies>
        </RollingRandomAccessFile>
        <Console name="STDOUT" target="SYSTEM_OUT">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="INFO">
            <AppenderRef ref="RFILE" level="INFO"/>
           <AppenderRef ref="STDOUT" level="INFO"/>
        </Root>
    </Loggers>
</Configuration>
```

log4j.xml

```
<Configuration status="INFO" monitorInterval="0" name="log4j2-config">
    <Appenders>
        <RollingRandomAccessFile name="RFILE" fileName="${sys:catalina.base}/webapps/my-logs/sigir-webapp.log"
                                 filePattern="${sys:catalina.base}/webapps/my-logs/$${date:yyyy-MM}/sigir-servlets-%d{yyyyMMdd}-%i.log.gz">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="250 MB"/>
            </Policies>
        </RollingRandomAccessFile>
        <Console name="STDOUT" target="SYSTEM_OUT">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="INFO">
            <AppenderRef ref="RFILE" level="INFO"/>
            <AppenderRef ref="STDOUT" level="INFO"/>
        </Root>
    </Loggers>
</Configuration>
```

Paths and names of the files

Appender to file

log4j.xml

```
<Configuration status="INFO" monitorInterval="0" name="log4j2-config">
    <Appenders>
        <RollingRandomAccessFile name="RFILE" fileName="${sys:catalina.base}/webapps/my-logs/sigir-webapp.log"
                                 filePattern="${sys:catalina.base}/webapps/my-logs/$${date:yyyy-MM}/sigir-servlets-%d{yyyyMMdd}-%i.log.gz">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="250 MB"/>
            </Policies>
        </RollingRandomAccessFile>
        <Console name="STDOUT" target="SYSTEM_OUT">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="INFO">
            <AppenderRef ref="RFILE" level="INFO"/>
            <AppenderRef ref="STDOUT" level="INFO"/>
        </Root>
    </Loggers>
</Configuration>
```

Policies define when create new files

log4j.xml

```
<Configuration status="INFO" monitorInterval="0" name="log4j2-config">
    <Appenders>
        <RollingRandomAccessFile name="RFILE" fileName="${sys:catalina.base}/webapps/my-logs/sigir-webapp.log"
                                 filePattern="${sys:catalina.base}/webapps/my-logs/$${date:yyyy-MM}/sigir-servlets-%d{yyyyMMdd}-%i.log.gz">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="250 MB"/>
            </Policies>
        </RollingRandomAccessFile>
        <Console name="STDOUT" target="SYSTEM_OUT">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="INFO">
            <AppenderRef ref="RFILE" level="INFO"/>
            <AppenderRef ref="STDOUT" level="INFO"/>
        </Root>
    </Loggers>
</Configuration>
```

Appender to console

log4j.xml

```
<Configuration status="INFO" monitorInterval="0" name="log4j2-config">
    <Appenders>
        <RollingRandomAccessFile name="RFILE" fileName="${sys:catalina.base}/webapps/my-logs/sigir-webapp.log"
                                 filePattern="${sys:catalina.base}/webapps/my-logs/$${date:yyyy-MM}/sigir-servlets-%d{yyyyMMdd}-%i.log.gz">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
            <Policies>
                <TimeBasedTriggeringPolicy />
                <SizeBasedTriggeringPolicy size="250 MB"/>
            </Policies>
        </RollingRandomAccessFile>
        <Console name="STDOUT" target="SYSTEM_OUT">
            <PatternLayout>
                <Pattern>%date{DEFAULT} %level [%thread] %class{1}.%method(%file:%line)%n\tIP = %MDC{IP}; USER = %MDC{USER}; ACTION = %MDC{ACTION}; RESOURCE = %MDC{RESOURCE}%n\t%message%n\t%throwable%n</Pattern>
            </PatternLayout>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="INFO">
            <AppenderRef ref="RFILE" level="INFO"/>
            <AppenderRef ref="STDOUT" level="INFO"/>
        </Root>
    </Loggers>
</Configuration>
```

Patterns of the output
(can be different depends on what you want)

log4j.xml

Appenders → specify where to “append” (write) the logs

Loggers → specify the appenders and the log levels that they must “append” (consider)

Logging and docker

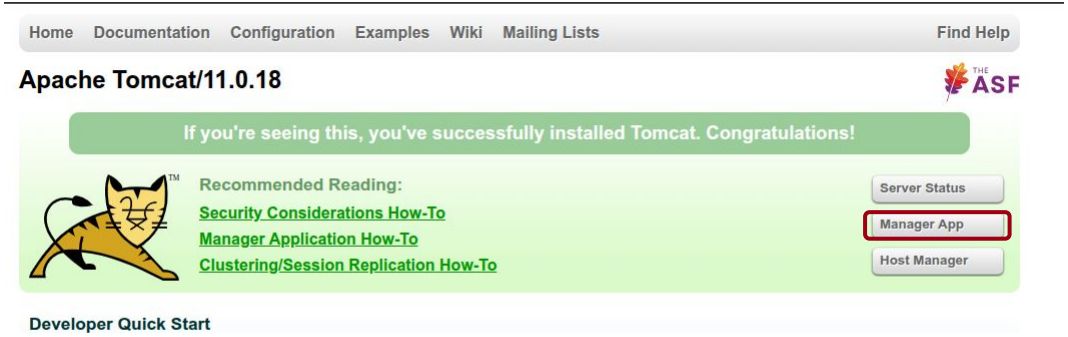
By default the **logs are stored within the container**.

To be able to see them, we need a **volume to map the logs** stored in the container to our local pc.

```yaml
services:


  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
```

Deploying the Web App

Deploying on Tomcat

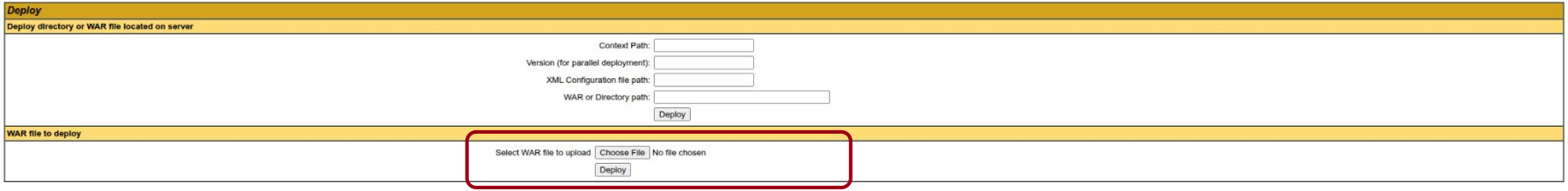


Once installed and started tomcat you can go to the address http://localhost:8080 and click on Manager App

After that you need to insert the credentials you set on tomcat-users.xml

Deploying on Tomcat

Then you can deploy your web application by selecting the desired .war file and clicking deploy.



Tomcat on Docker

When we use **Tomcat through docker** we map our .war file using **volumes**!

```yaml
services:

  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
```

Tomcat on Docker

However, if we apply some modifications to our web app and we **re-package it using MAVEN** the application is not automatically re-deployed.

We can solve this by mapping a new configuration file that enables auto-deployment.

```
services:


  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
```

Tomcat on Docker

If you need to access Tomcat but you use docker you need to add volumes to:

1. Allow Tomcat to be seen outside the container

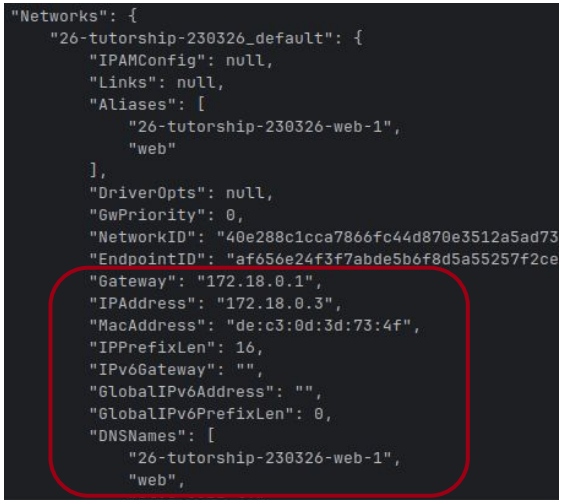
2. Set the users credentials

```
services:



  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps.dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
```

Tomcat on Docker

The internal address of tomcat typically starts with 172, however, you can check it with the command:

docker inspect <container-id>



tomcat-users.xml →
same as the “local” one
(see Lecture 2 tutorship)

Tomcat on Docker

ATTENTION: after adding the volumes and having deployed (docker compose up) our container, we need to execute the following commands:

docker exec -it <container-id> bash

and

cp -r webapps.dist/* webapps/

From DB to Servlets

Servlets: Recap

A servlet is a **Java technology-based Web component**, managed by a container, that **generates dynamic content.**

Servlets defines a **server-side API for handling HTTP requests and responses.**

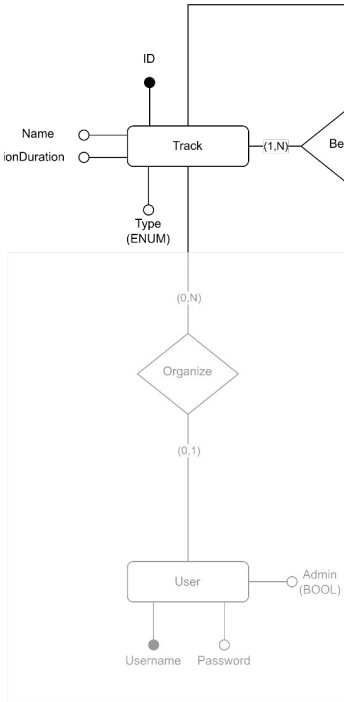
Servlets are **platform-independent Java classes that are compiled to platform-neutral** byte code that can be loaded dynamically into and run by a Java based Web server.

To handle the requests, the developer must make adequate provisions for concurrent processing with multiple threads (**Not thread safe**).

SIGIR WebApp - Users Functionalities

In the web app for SIGIR we have users which must be able to:

- **Signup**: register to have full access to the sigir web app
- **Login**: users that are already registered can login into the web app and access restricted information
- **Search**: e.g. list all the other users of a track (e.g. imagine an admin that needs to manage all the users)



SIGIR WebApp - Key Concepts

Key elements when you develop a web application:

- Clear project structure – *follow* ***(DON'T JUST COPY)*** course exs
- Ensure **dependencies consistency** in pom
- **Map servlets** in web.xml
- Configure **logs**
- Understand the **resources involved**
- **Configure database connection**
- Discuss the operations to be performed and write the corresponding servlets

SIGIR WebApp - Resources

Resources are reusable software components which can be manipulated by the web application.

We have 3 resources:

- User
- Track
- Message

Each resource has some fields accessible via accessor methods such as getXXX or setXXX.

Resources

Messages notify us about possible errors or information. For messages we define two constructors:

- Error messages
- Informative messages

Users have the following fields:

- Username, Password, Admin (boolean flag) and Track_ID

Tracks have the following fields:

- ID, name, presentation_duration, type (enumeration)

Connect to the DB

1. Write the context.xml file
   The context.xml is a configuration file defines a connection pool that your application can use to connect to the database. Basing on the professor's examples change the following elements: **Name, url, username, password according to your postgres configuration**
   Remember that if you run a docker container the URL must not contain localhost!! It must contain the **docker service name** instead (i.e., db) followed by the port you are mapping to.

2. In the web.xml add the <resource-ref> where you report the resource name declared in the context.xml

DAOs

1. In the dao folder, put the AbstractDAO and the DataAccessObject interface. They should not be modified. Put these class and interface in your project and **extend/implement** them (**DO NOT LEAVE THEM AS THEY ARE**).

2. The AbstractDAO implements the DataAccessObject interface. In this way, all the subclasses have a uniform behaviour.

3. The DataAccessObject interface encapsulates all the logic to access the database.

4. **Define a DAO for each access operation**
   → If we have to search some users, insert new users, modify users' data, delete an user, we will have 4 DAOs.

SIGIR WebApp - DAOs

1. **SearchUserDAO** → allows to search for a user given an attribute

2. **SearchTrackDAO** → allows to search for a track given an attribute

3. **InsertUserDAO** → allows to insert a new user.

DAO - Transactions

A DAO might contain multiple statements which can involve different tables.

Ex. I want to create a new user together with a new track → I have to add the track in the track table, the users in the users table and **“link them”**.
But, if something strange happens – i.e., connection goes down – I don’t want that the track is added but its user(s) are not.

I want to execute these two operations in a single transaction (**multiple operations executed as a single unit**) which rolls back if needed.

Here you can find an example:

https://bitbucket.org/frrncl/tutor-2023/src/master/group_creation_rest_17042023/src/main/java/GroupCreation/dao/CreateGroupDAO.java

SIGIR WebApp - Servlets

We have 2 servlets: one for login/signup, one for listing users.

ListUsersServlet:

- List Users of a track

LoginServlet:

- Login User
- Signup User

List Users

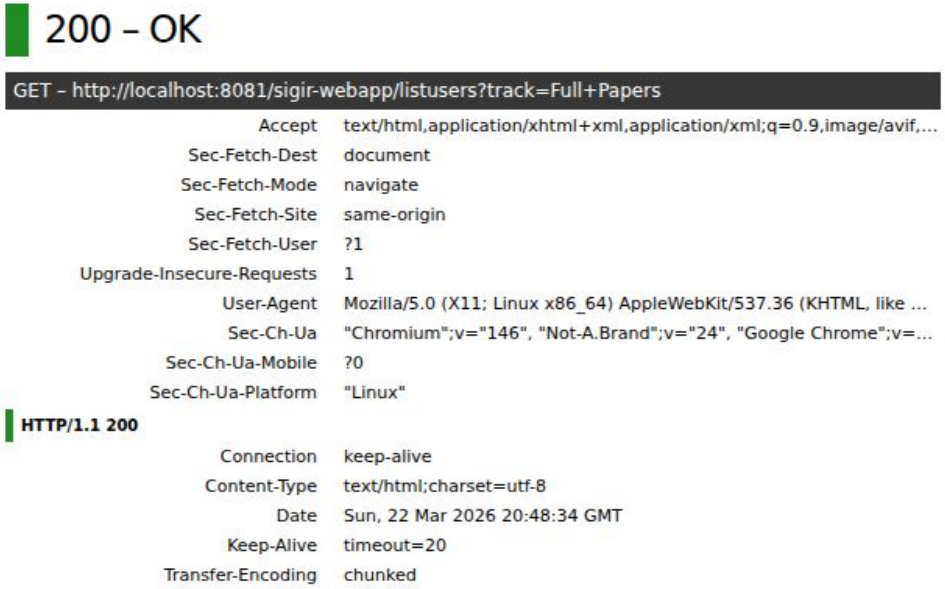
You must have clear what are the information users must provide, and which are the constraints related to them.

The provided information must be **consistent with the data type and attributes provided in the database** otherwise a lot of errors can occur!

SIGIR Web App: GET Request

Ex.
List users of track
“Full Papers”

GET Request



LOGIN

We have two possibilities to log the user in:

- **GET** Request

- **POST** Request

In both cases we can exchange information with the server.

SIGIR WebApp - Login and Signup

When you have to develop login and signup servlets, remember that these operations are always POST requests, hence their implementation will be in the doPost.

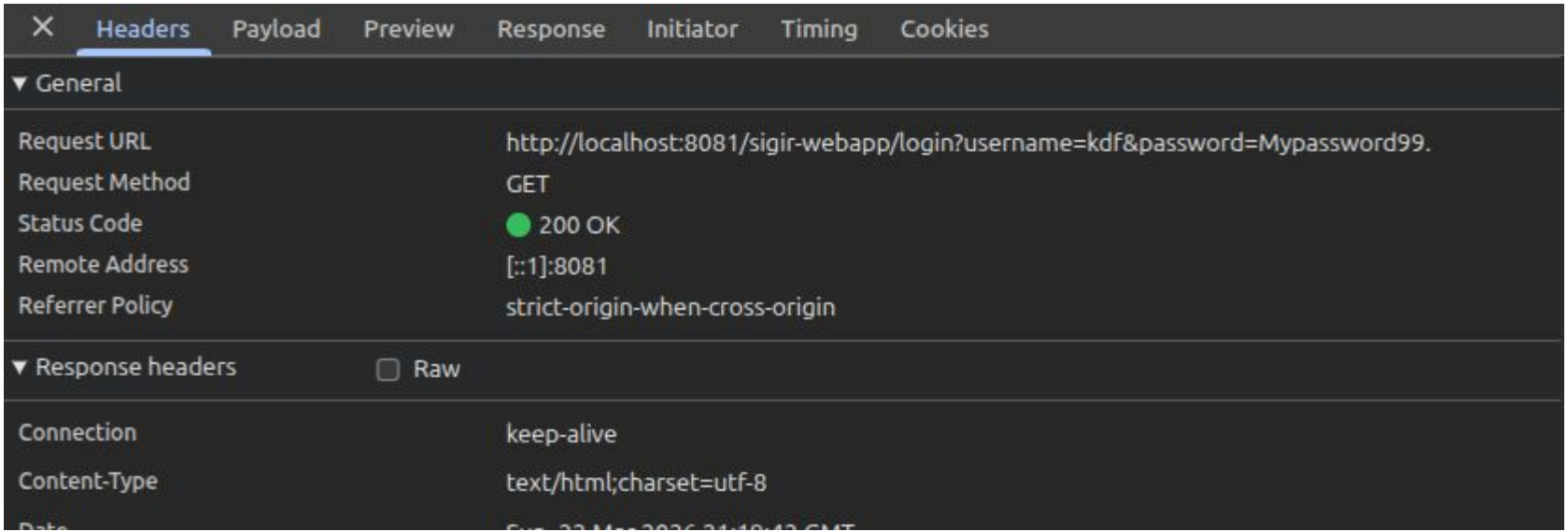
When users provide their data to login or signup, you must pay attention to the following key concepts:

- usernames are always in lower case: **it is better to turn these fields to lower before adding new users** in the db or check the user's existence.

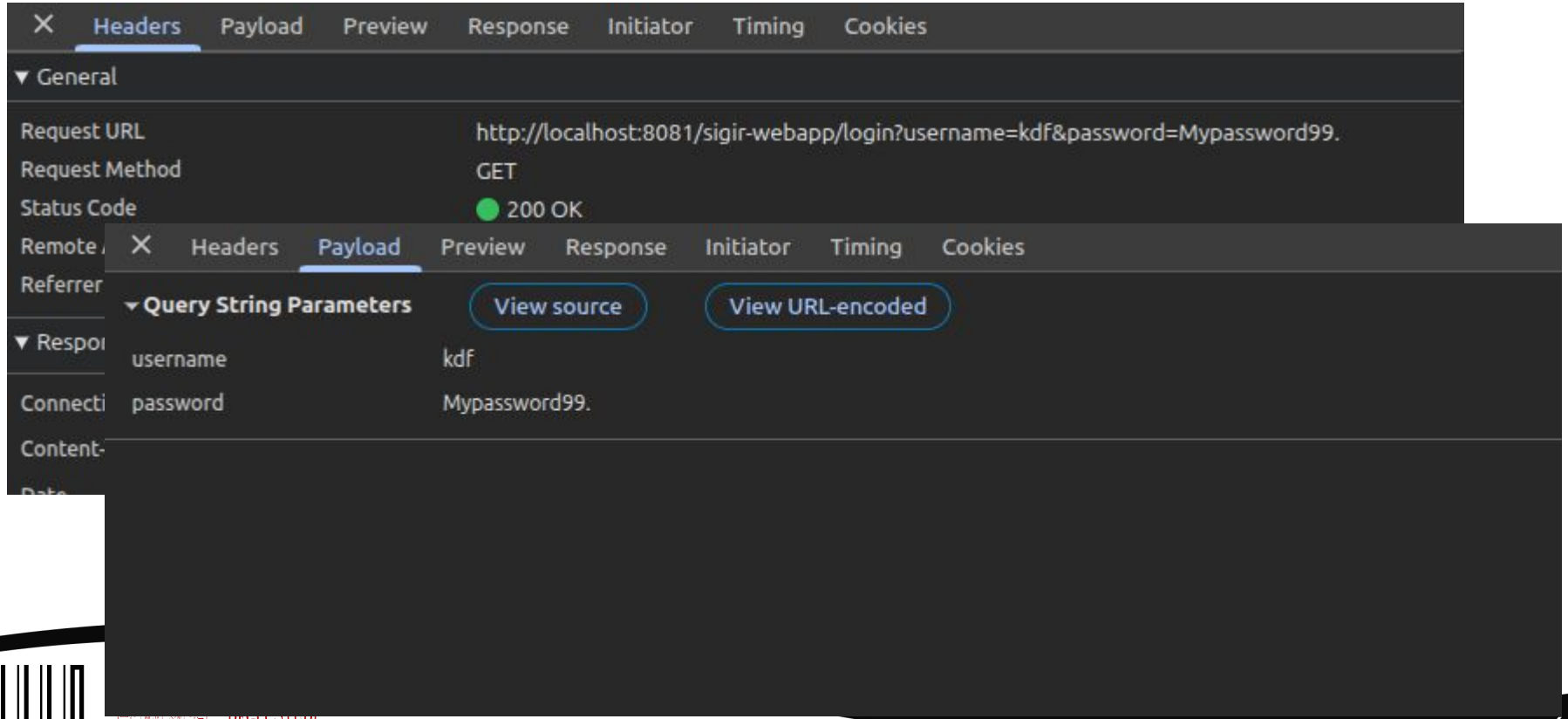
- Before login a user, always check that all the parameters you required to login are provided – i.e., **always check that a password is provided**.

- Check that all the **parameters comply with your requirements**: always check that passwords contain at least 8 characters for example.

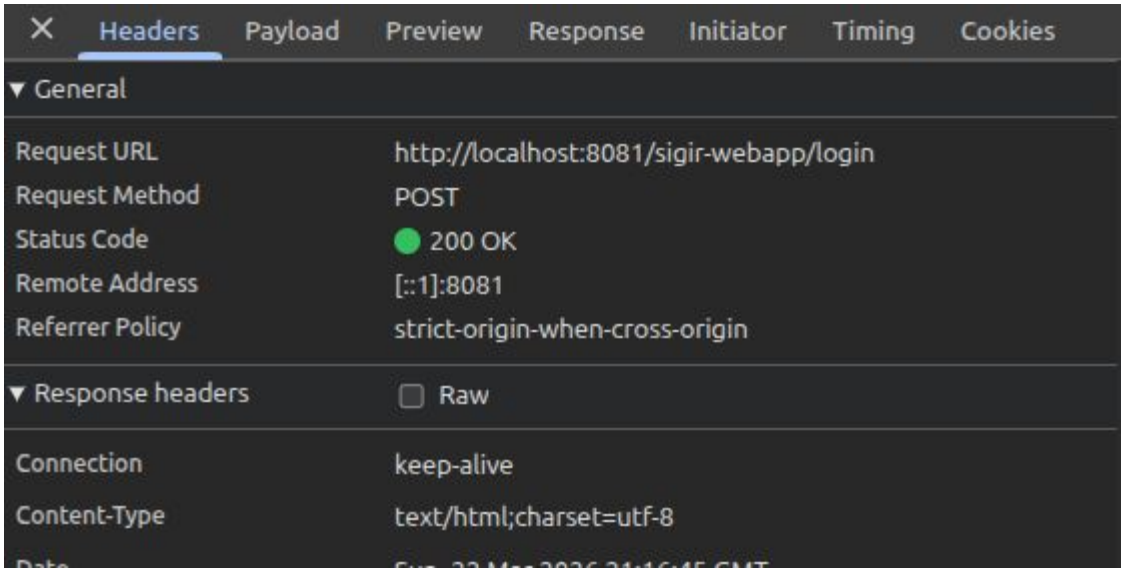
SIGIR WebApp - Login w/ GET



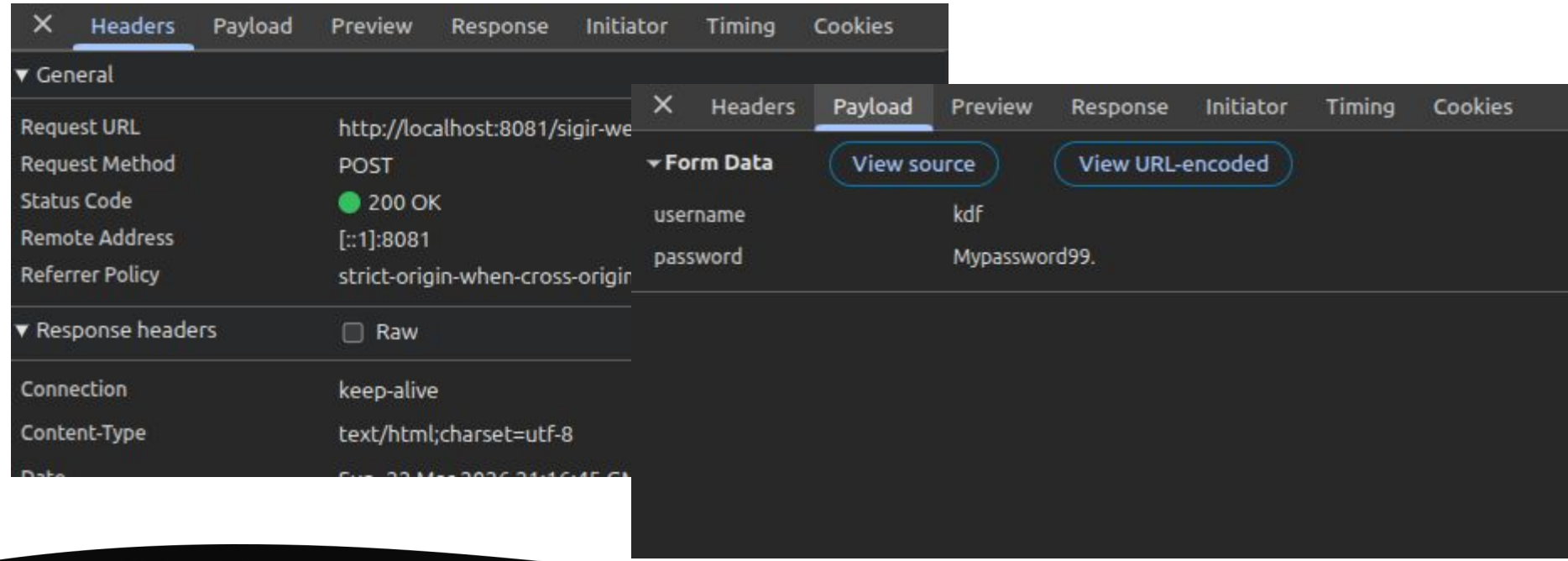
SIGIR WebApp - Login w/ GET



SIGIR WebApp - Login w/ POST



SIGIR WebApp - Login w/ POST



How can I choose between GET and POST?

A **GET request is used to retrieve data** from the database. When a client sends a GET request, the *parameters* are *included in the URL as a query string*.

A **POST request is used to send data to the server** to create a resource. When a client sends a POST request, the *data* is *included in the body of the request*.

GET requests are less secure than POST requests because the parameters are **passed as part of the URL**, which can be seen by anyone who has access to the URL. POST requests, on the other hand, hide the parameters in the request body, which is not visible in the URL.
→**LOGIN INCLUDES SENSITIVE INFORMATION ABOUT A USER! WE CAN NOT PASS A PASSWORD AS A QUERY STRING**

How can I choose between GET and POST?

A **GET request is used to retrieve data** from the database. Wh
GET request, the parameters are included in the U

A **POST request is used to se**
client sends a PO

MORE ON THIS DURING WEB SECURITY LECTURE

as access to the
parameters in the request body,

TIVE INFORMATION ABOUT A USER! WE CAN NOT PASS
D AS A QUERY STRING

SIGIR WebApp - HTML Forms

We have to define 3 forms:

- **Login**: asks for username and password: these two fields are REQUIRED hence users must provide them, otherwise we cannot login them
- **Signup**: asks for username, password, admin, track id. The admin flag is optional, if not provided the user is not an admin.
- **Passwords** must contain at least 8 chars, one digit and one special char
- **Search form**: we ask for a track name

The User's Inputs

User is required to insert his username and a password. How can we validate if these inputs exist, and are correct?

1. In the **HTML page** → required attribute makes sure that the user added a value and the form is not submitted if all the required inputs are not filled. In addition, the type of input may define how the input should be formatted (e.g., email).

The most typical (and expected) behaviour of a user is: accessing our web application and adding his credentials directly from the form we created.

But this is not the only way. Servlets can not be called exclusively from the html pages. **Servlets are called depending on the URI we request, the type of request, and the payload**. We can bypass frontend validation applied to the HTML inputs and reach servlets which process the parameters we sent.

The User's Inputs

Pay attention to this behaviour → **ALWAYS PUT A CHECK ALSO IN YOUR SERVLETS!!**

Avoiding checking the request parameters in the servlets won't throw any exception, and your web application will seem to correctly work.

In some cases this might be dangerous, for example users who are not 18 may have access to some forbidden content and some actions could be erroneously allowed (for example: the creation of a bank account).

Useful Tips - CURL

CURL allows you to issue requests to the web server. You can perform any kind of operations bypassing the frontend.

This means that, for example, if you forget to check if the user provided a password from your servlet, and you checked only in the html, **anyone can have access to other users' data and information**.

This Is dangerous and might cause the loss of your data.

→ ALWAYS CHECK IF YOUR SERVLETS WORK USING (ALSO) CURL

Ex. of using curl

```
~/IdeaProjects/tutoring-WA/26-TA/26-tutorship-230326 curl http://localhost:8081/sigir-webapp/login\?username\=kdf\&password\=Mypassword99.
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Login</title>
</head>
<body>
<h1>LOGIN USER - SUCCESS</h1>
<hr/>
<p>Login success</p>
<ul>
<li>username: kdf</li>
<li>admin: false</li>
<li>trackID: 1</li>
```

Final Remark on HW1

HW1 - Presentation Layer

In Homework 1, the implementation of the presentation layer is not required. By the way, you should have a clear idea of:

- **How each web page will look like**
  Which information should the page contain? Are there any elements which are shared with other web pages?
- **What are the interactions that a user can perform on that page?**
  Are there buttons? What happens clicking on them? Are there forms? Should they be automatically filled so that a user can change its content? What happens if some errors happen? How the error messages are displayed? How errors are handled from the interface point of view?
- **Who has access to the web page**
  Are there some web contents which should not be available to all the users of the web applications?
  In the SIGIR WebApp for example, only admins should have access to all pages (e.g. organizing sessions)